this case you'll use the ES256 signature algorithm (ECDSA with SHA-256), which is widely implemented.

Listing 11.1 Generating a JWT client assertion

```
import java.util.*;
import com.nimbusds.jose.*;
import com.nimbusds.jose.crypto.ECDSASigner;
import com.nimbusds.jwt.*;
import static java.time.Instant.now;
import static java.time.temporal.ChronoUnit.SECONDS;

var clientId = "test";
var as = "https://as.example.com/access_token";
var claims = new JWTClaimsSet.Builder()
        .subject(clientId)    #A
        .issuer(clientId)    #A
        .expirationTime(Date.from(now().plus(30,
SECONDS)))    #B
        .audience(as)    #C
        .jwtID(UUID.randomUUID().toString())    #D
        .build();
var header = new JWSHeader(JWSAlgorithm.ES256);    #E
var jwt = new SignedJWT(header, claims);    #E
jwt.sign(new ECDSASigner(privateKey));    #E
var assertion = jwt.serialize();    #E
System.out.println("Assertion: " + assertion);
```

#A Set the subject and issuer to the client ID
#B Add a short expiration time
#C Set the audience to the token endpoint URI
#D Add a random JWT ID
#E Sign the JWT with the private key